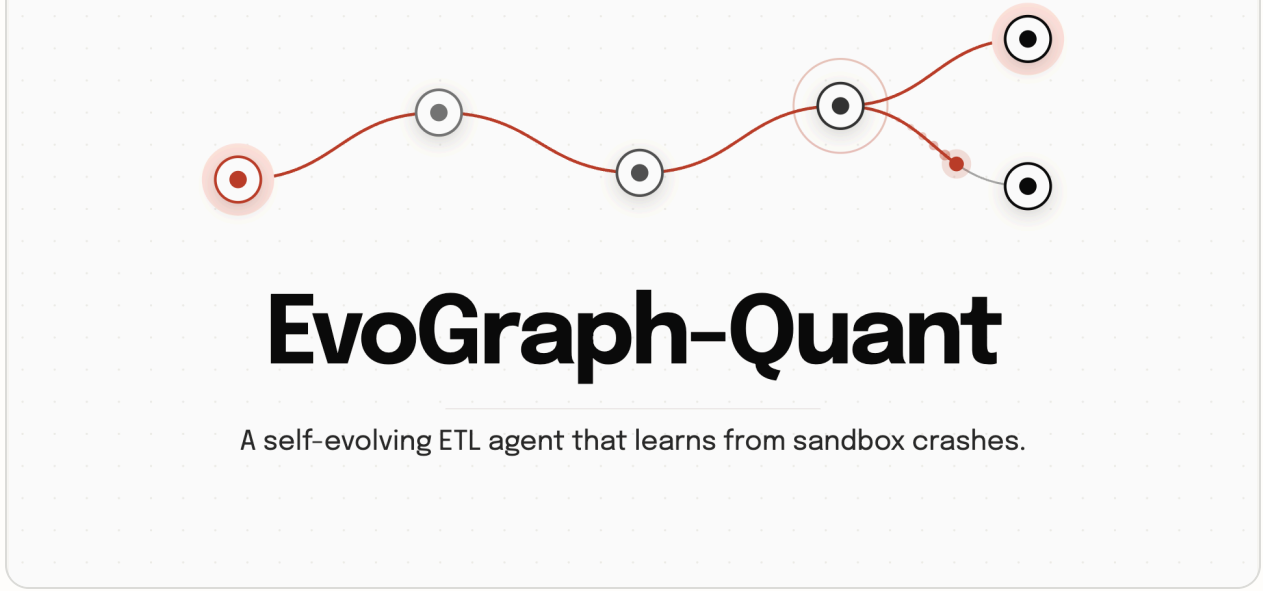
EvoGraph-Quant

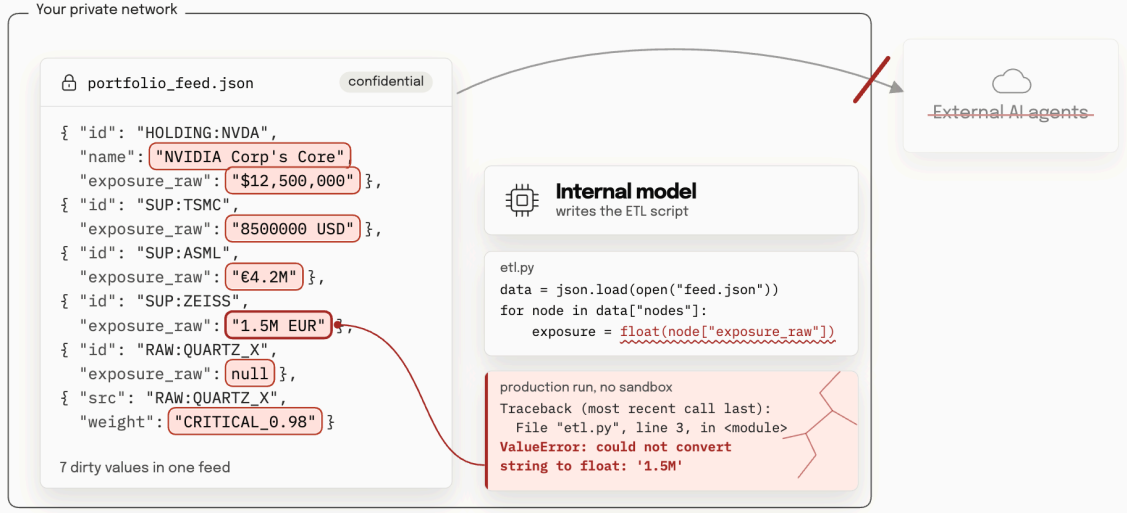
A self-evolving ETL agent that learns from sandbox crashes, so only verified data reaches the risk graph.

Daytona NOSANA neo4j



Risk teams cannot hand this data to an outside agent.

The task	Turn confidential holdings and alternative supply-chain intelligence into one risk graph.
The rule	The data stays inside. No closed, external coding agents.
The gap	Small in-house models write the ETL, and it breaks on the first dirty field.



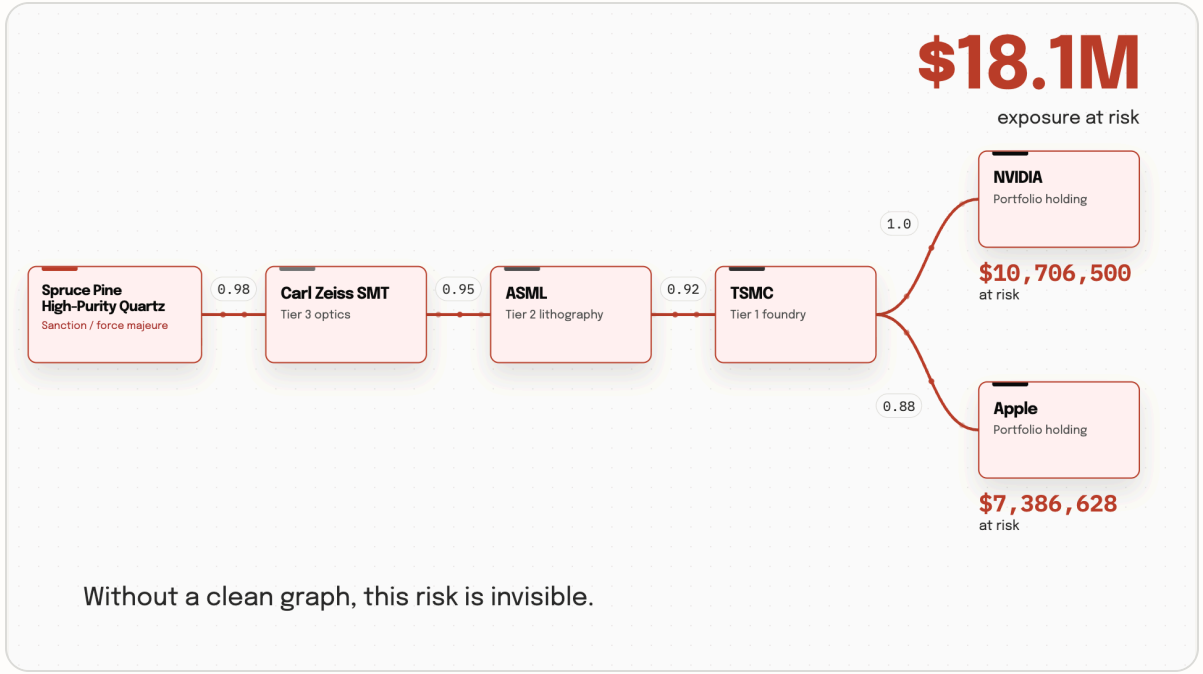
Private models write fragile ETL. It breaks on dirty data.

One dirty field can hide an $18.1M exposure.

`'$12,500,000'` `'€4.2M'` `'1.5M EUR'` `null`

`'CRITICAL_0.98'` `"NVIDIA Corp's Core"`

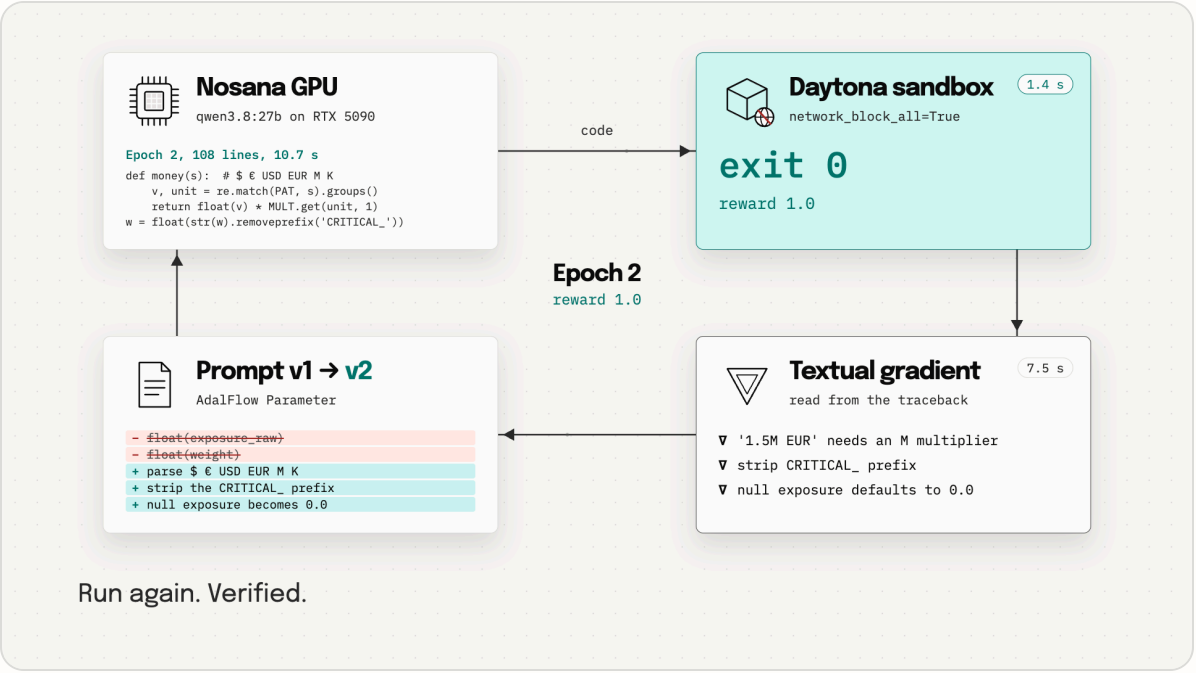
1. Dirty values arrive in the feed
2. The ETL crashes, or writes wrong numbers silently
3. The risk graph has holes
4. Multi-hop contagion becomes invisible
5. **A quartz outage reaches NVDA and AAPL unseen**



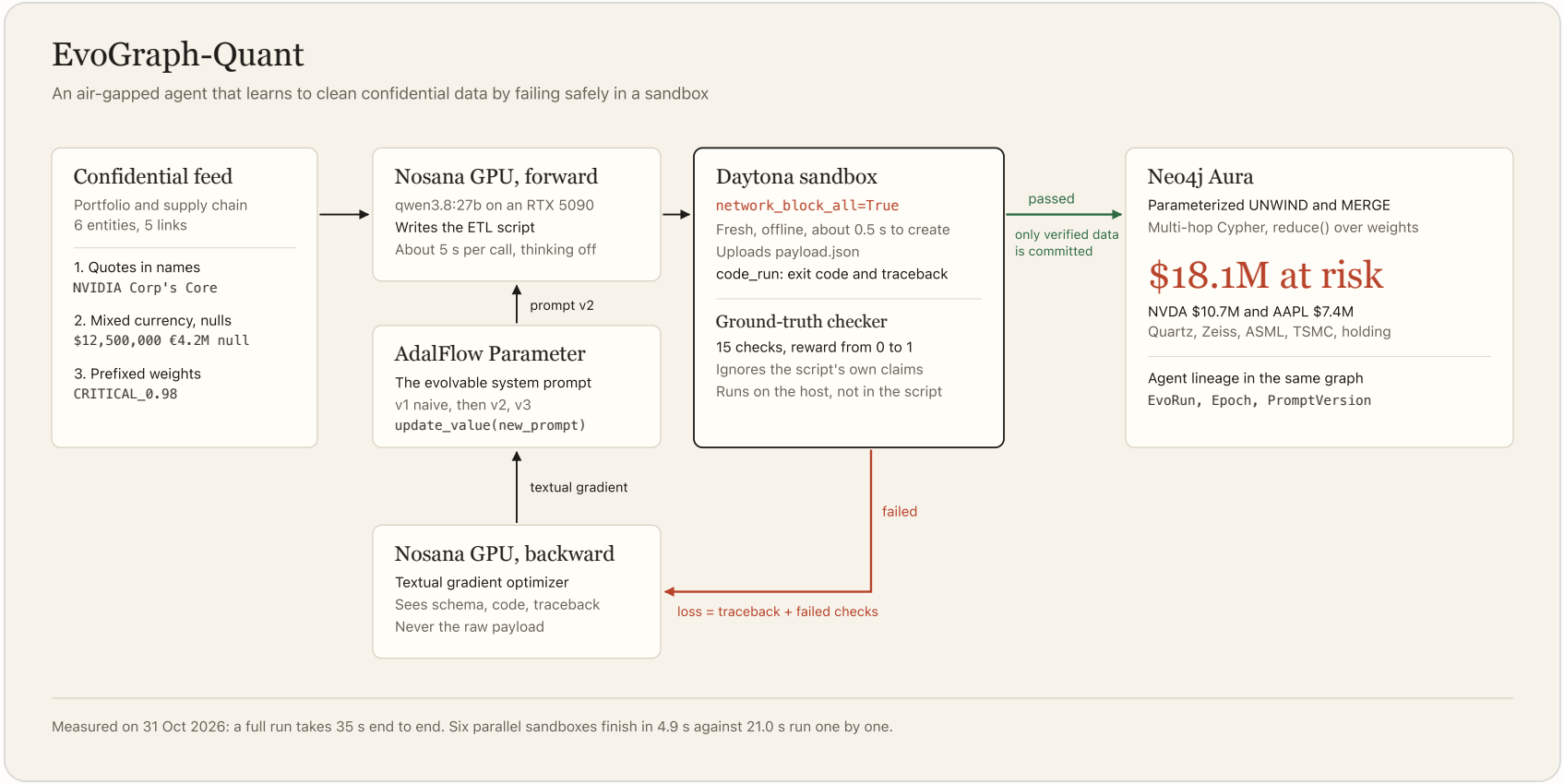
Without a clean graph, this risk is invisible.

Treat the sandbox as the loss function.

Loss	Run the generated script and keep its exit code and output.
Gradient	The traceback plus every failed check, written as text.
Parameter	The system prompt, rewritten each epoch.
Commit rule	Only rows that pass the ground-truth check reach Neo4j.



The architecture, end to end.



NOSANA generate and optimise

Both passes run on a private Nosana GPU.

```
chat("forward", msgs)                  # writes the ETL script
chat("backward", msgs, json_mode=True) # gradient + new prompt
# qwen3.8:27b, Ollama /v1 on an RTX 5090 job
```

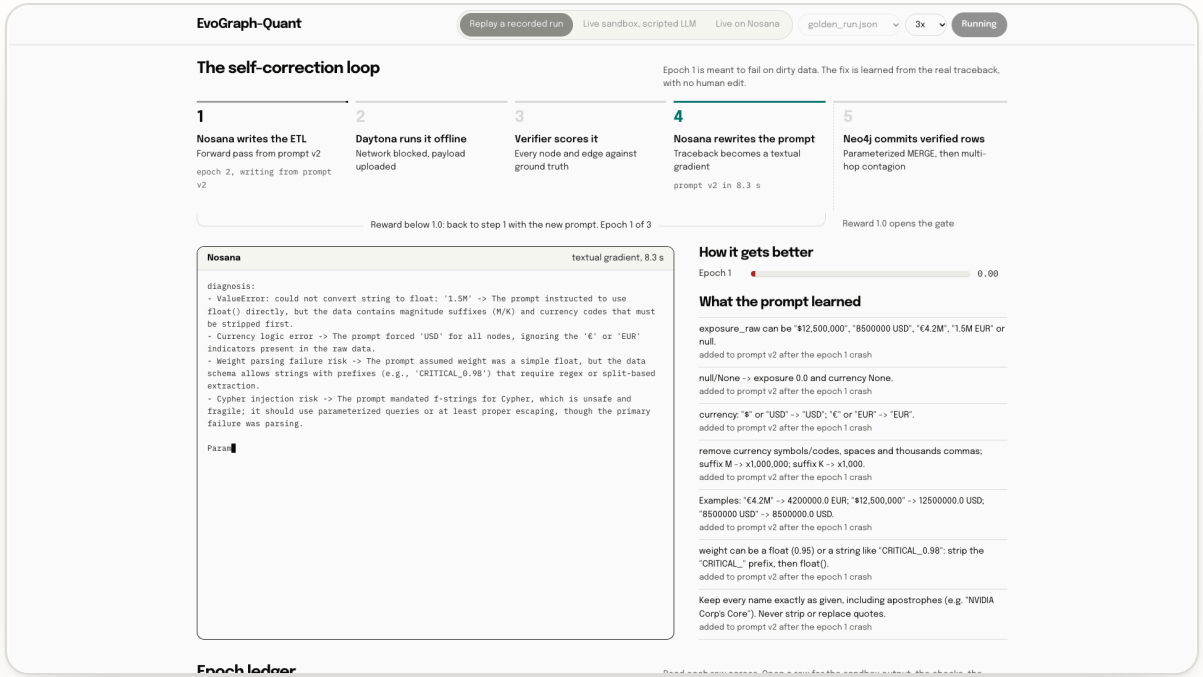
5.4 s

a 101-line ETL script

5.1 s

a JSON prompt rewrite

The public endpoint failed on a read-only model mount, so requests reach the same GPU over Nosana's SSH channel. Data passes only the Nosana node and our laptop.



Daytona execute, and the result is the loss

Every script runs in an offline Daytona sandbox.

```
sb = daytona.create(CreateSandboxFromSnapshotParams(
        language="python", network_block_all=True))
sb.fs.upload_file(payload, "payload.json")
r = sb.process.code_run(code, timeout=60) # exit_code + traceback
```

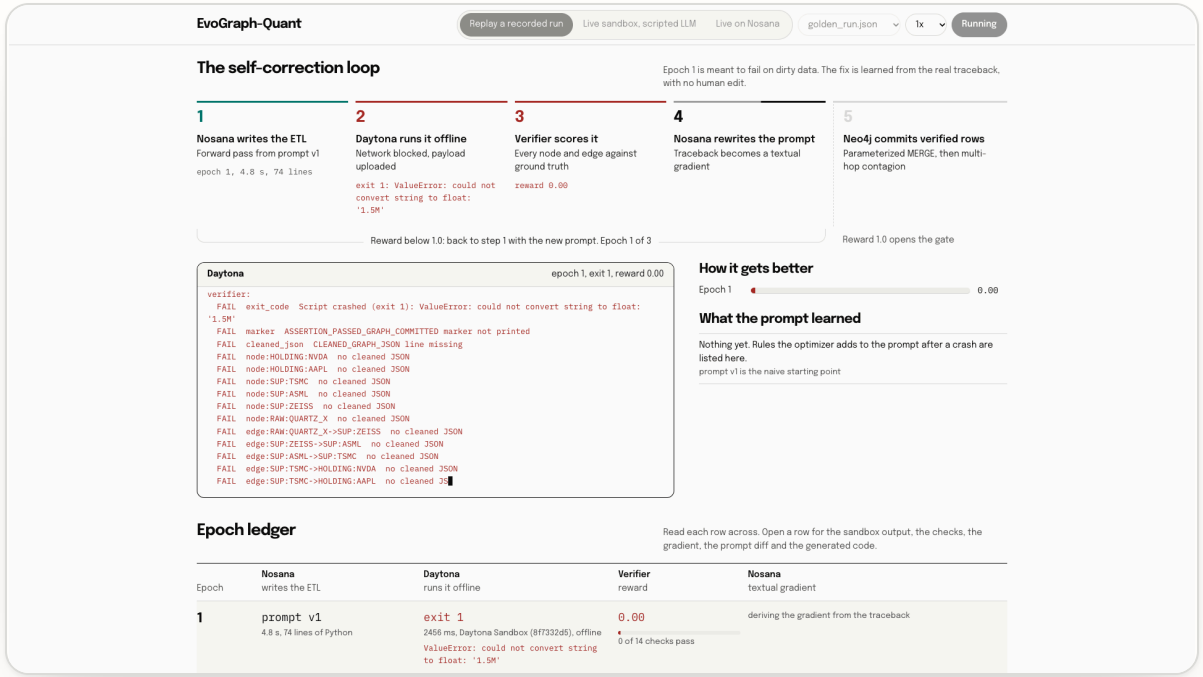
0.5 s

to create a sandbox

4.3×

6 sandboxes in parallel: 4.9 s against 21.0 s

Inside the sandbox, Aura and pypi.org both fail DNS: the generated code cannot leak the feed.



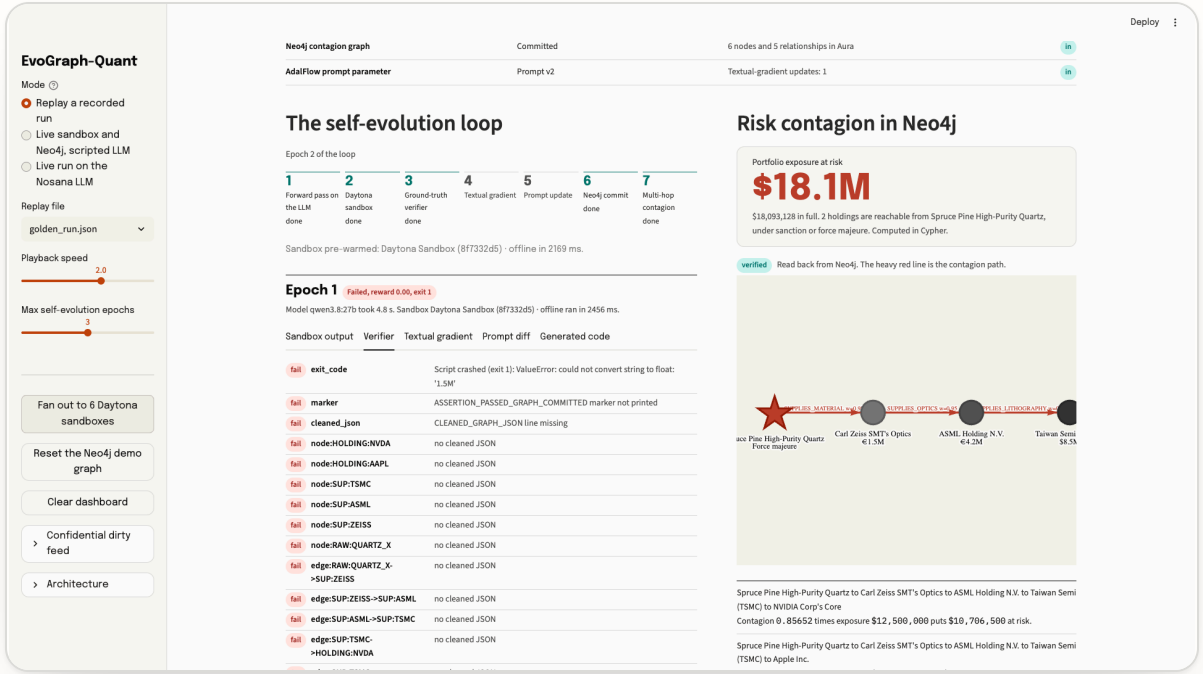
sandbox_verifier.py, parallel_sandbox.py

We compute the reward. The script never claims it.

15 checks	Exit code, marker, JSON, six nodes, five edges, nothing extra.
Partial credit	Reward is the share of checks passed.
No shortcut	Printing the success marker without correct data still fails.

0.733

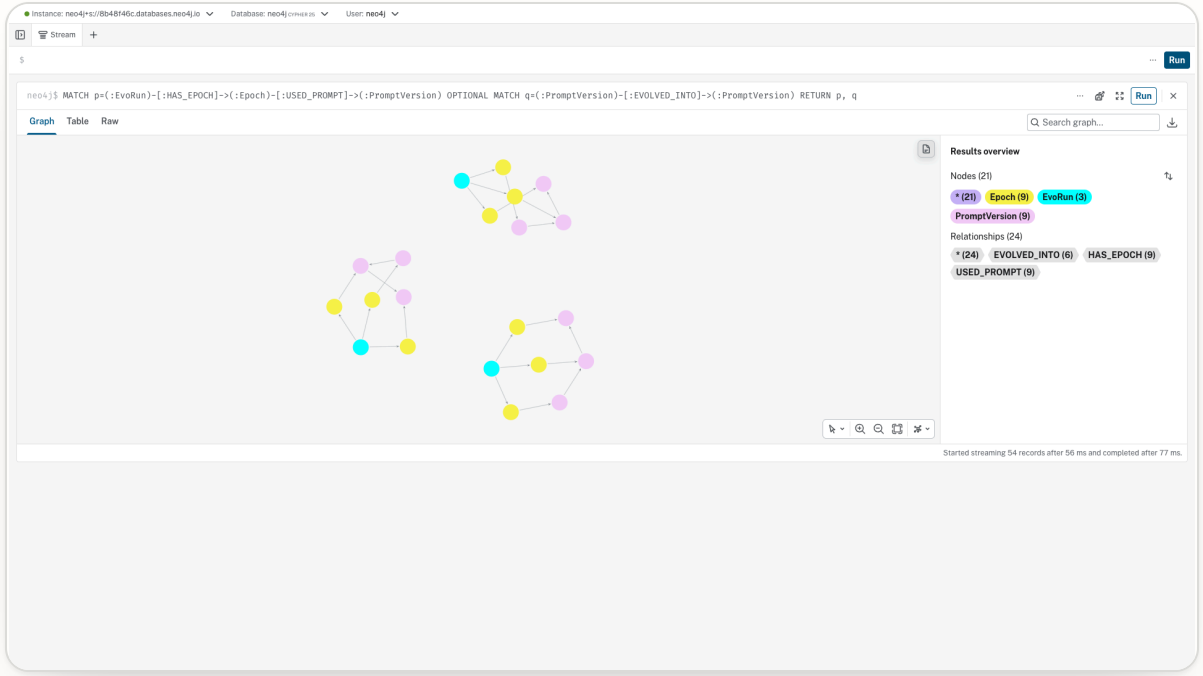
a half-fixed script: currencies right, the M multiplier missed on ASML and ZEISS



The prompt is the parameter; the traceback is its gradient.

Store	An AdalFlow Parameter holds the prompt, one version per epoch.
Privacy	The backward pass sees the schema, the code and the error, never the raw portfolio.

```
# gradient, written by qwen3.8:27b
The data contains magnitude suffixes (M/K) and currency codes that
must be stripped and multiplied before float conversion.
```



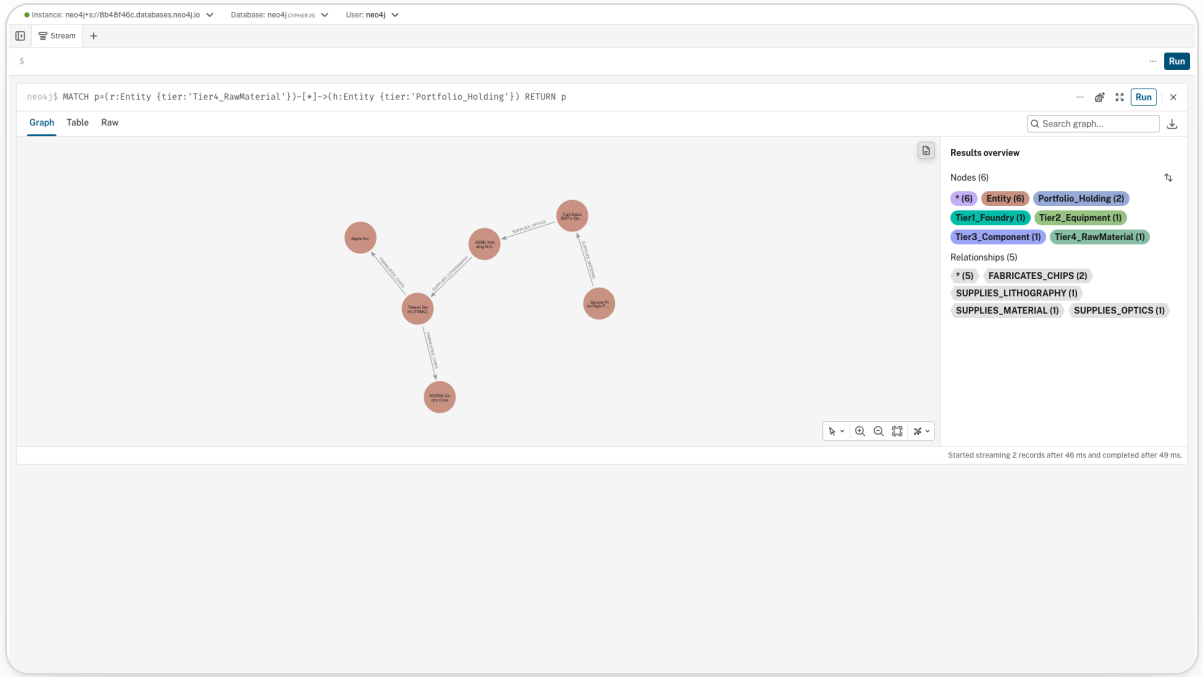
commit, reason, remember

Neo4j holds only verified rows, and computes the contagion.

```
UNWIND $rows AS row MERGE (n:Entity {id: row.id})
MATCH p = (src:Entity {tier: $source_tier})-[*1..8]->(h)
WITH p, h, reduce(c = 1.0, r IN relationships(p) | c * r.weight)
AS contagion
```

Holding	Contagion	At risk
NVDA	0.85652	$10,706,500
AAPL	0.7537	$7,386,628

Query time 156 to 241 ms. The lineage of every prompt sits in the same graph.



Integration depth.

Daytona

Execute, and the result is the loss

7 capabilities

Offline create with network_block_all
fs.upload_file and process.code_run
Warm reuse, delete and label cleanup
Parallel fan-out, best of K

NOSANA

Generate and optimise

6 capabilities

RTX 5090 GPU job, Ollama OpenAI API
Job SSH channel relay
Forward pass and JSON backward pass
Prompt lab, 5 runs per prompt

neo4j

Commit, reason, remember

7 capabilities

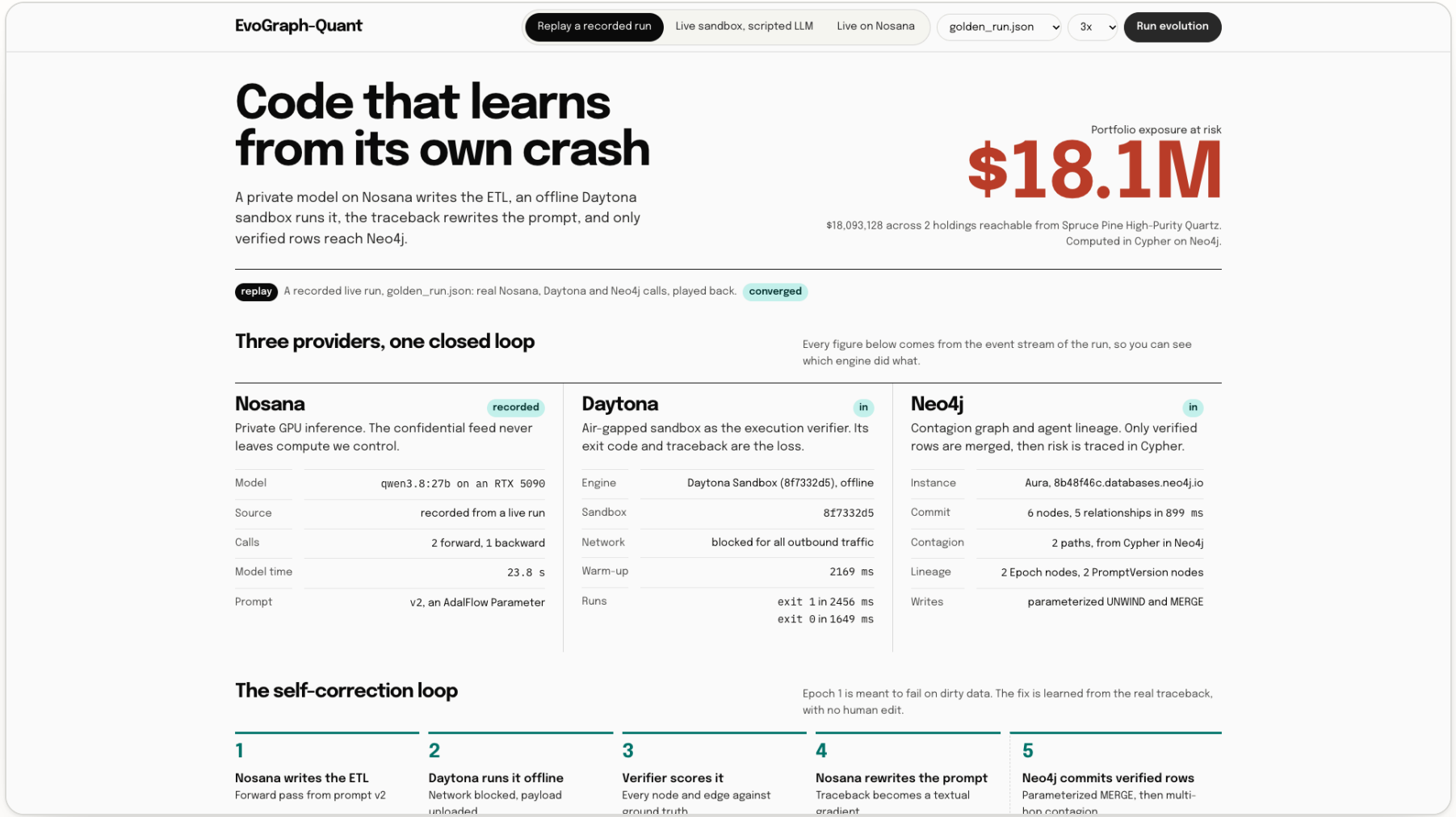
Constraints, UNWIND and MERGE
Label and type whitelist
Variable-length paths with reduce()
Lineage graph, read-back for the UI

One recorded run took 35 seconds, end to end.

Step	Result	Time
Sandbox warm-up	offline Daytona sandbox ready	`2.2 s`
Epoch 1	`ValueError: could not convert string to float: '1.5M'`	`4.8 s + 2.5 s`
Textual gradient	prompt v2 written on Nosana	`8.3 s`
Epoch 2	**reward 1.0, verified**	`10.7 s + 1.6 s`
Neo4j commit	6 nodes, 5 relationships	`0.9 s`

The 27B model cleaned half the data on its own, then crashed on '1.5M EUR'.

The live product.



Live with Nosana, live with a scripted LLM, or replay; the graph is read back from Neo4j

One backward pass turns 0 of 5 into 5 of 5.

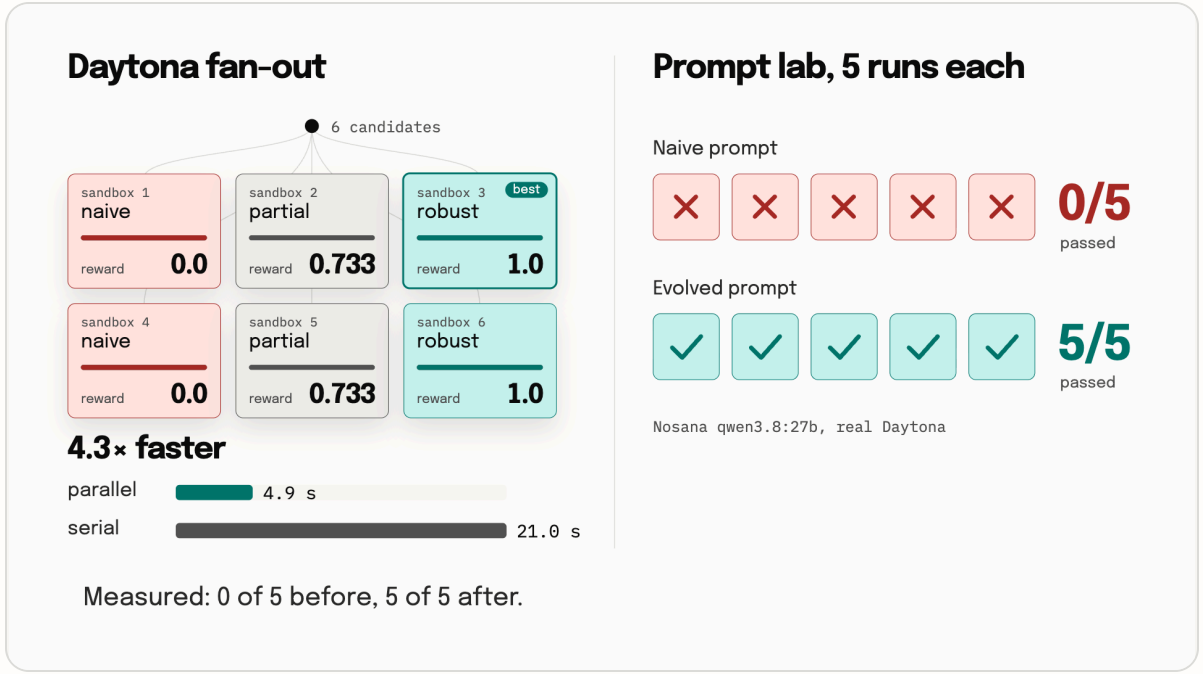
0 / 5

naive prompt, every run exits 1

5 / 5

after one backward pass, reward 1.0 every run

Five runs each on qwen3.8:27b with real Daytona. Stable in practice, not a statistical claim.



New dirt becomes the next loss.

New feed	EV battery chain, 15 nodes. Our final script crashes on an unseen `HIGH_` prefix: reward 0.0.
One rule later	Reward 0.946. Only the yen sign is left unmapped.

Live mode calls Nosana, Daytona and Neo4j for real.

Replay files are recorded runs and are labelled as such.

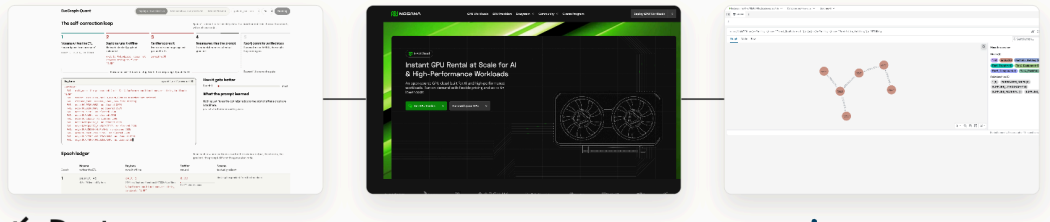
A Claude stand-in answers only if Nosana is down, and the screen says so.

What comes next.

Daytona	A verified ETL snapshot, and volumes for large vendor feeds.
Nosana	Job-definition deploys, one GPU job per candidate prompt.
Neo4j	Passing scripts and their lineage as a reusable ETL library.

Live demo →

Built on Daytona, Nosana and Neo4j



Daytona
Executes every candidate offline
`0.4 s create, 4.3x fan-out`

NOSANA
Writes and rewrites the ETL
`qwen3.8:27b, 5 s per call`

neo4j
Commits only verified rows
`$18.1M at risk, in Cypher`

Three sponsors, one closed loop.